

Design Verification Report

RISC-V Pipelined CPU

Five-Stage Top-Level Integration Testbench

Design Author: Elliot Staresinic

DUT: pipelined_cpu_top_level | Stimulus: program.mem (30 instructions, 34 retired in 48 cycles)

Simulator: Icarus Verilog (iverilog -g2012)

Date: 2026-08-04 | Status: PASS

VERIFICATION RESULT: PASS (0 errors)



SILICON FROM SCRATCH

Contents

Contents	2
1. Executive Summary	4
2. Design Under Test	4
2.1 Supported Instruction Subset.....	4
2.2 System Architecture	5
2.3 Module Hierarchy	6
2.4 Pipeline Registers.....	6
2.5 Hazard and Forwarding Units	7
3. Verification Methodology and Testbench.....	8
3.1 Key Parameters.....	8
3.2 The Shadow Pipeline.....	8
3.3 Sampling Point	8
3.4 Oracle 1: Lockstep Golden Model.....	8
3.5 Oracle 2: Retirement Order.....	9
3.6 Oracle 3: Final-State Table	9
3.7 Pipeline Drain Check	9
3.8 Supported Instruction Subset of the Golden Model	9
3.9 Halt Condition and Watchdog	10
3.10 Waveform Dump.....	10
3.11 Per-Instruction Trace and Disassembler	10
3.12 Hierarchical References	10
3.13 Verification Task Reference.....	10
4. Program Under Test.....	12
4.1 Execution Trace.....	12
4.2 Cycle Accounting.....	13
5. Verification Results	14
5.1 Lockstep Check.....	14
5.2 Retirement Order.....	14
5.3 Pipeline Drain	14
5.4 Final-State Oracle.....	14
5.5 Final Register File.....	14
5.6 Final Data Memory (non-zero words).....	15
5.7 Pipeline Behavior.....	15
6. Functional Coverage	16
6.1 Coverage Matrix	16

6.2 Squashed Instructions	17
7. Comparison with the Single-Cycle Design.....	18
7.1 What Changed in the RTL	18
7.2 Architectural Equivalence	18
7.3 Cycle Count.....	18
7.4 Static Timing Analysis Flow	19
7.5 Measured Critical Path and Area	19
7.6 Runtime and Speedup	20
7.7 Sensitivity to the Memory Estimate	20
7.8 What the Flow Does Not Model	20
8. Issues and Recommendations	21
8.1 Open Issues	21
8.2 Recommendations.....	21
9. Conclusion	21
Appendix A. Simulation Waveform	23

1. Executive Summary

This report documents the functional verification of a five-stage pipelined RISC-V CPU at the top level (module `pipelined_cpu_top_level`), exercised with the same 30-instruction program that verified the single-cycle design and checked against three independent reference oracles.

Result: PASS, 0 errors. 34 instructions retired in 48 clock cycles before a clean halt, which is 1.41 cycles per retirement. Of the 14 cycles that retired nothing, three are the initial pipeline fill, seven are hazard stalls and four are branch flushes; Section 4.2 accounts for every one of them individually. Nothing committed during the eight-cycle drain that followed the halt, and the design terminated without watchdog intervention.

The architectural end state is identical to the single-cycle design's, register for register and memory word for memory word. That is the property pipelining has to preserve, and the final-state table in Section 5.6 is the same table the single-cycle report checked. Section 7 compares the two designs directly and reports the static timing analysis run over both: the pipeline holds a 1.92x clock advantage against the 1.41x it needs to break even on cycle count, so it finishes the program 1.36x faster for 33% more cell area.

2. Design Under Test

The DUT is a five-stage pipelined RV32I-subset processor using a Harvard architecture with separate instruction and data memories. The stages are IF, ID, EX, MEM and WB, separated by four pipeline registers. Branches resolve in the ID stage rather than in EX, so a taken branch squashes exactly one wrongly fetched instruction. Data hazards are resolved by forwarding wherever the value exists, and by stalling only where it does not yet. The register file writes on the negative clock edge, so a value written in WB is readable in ID in the same cycle.

The top level also carries four `dbg_*` observability outputs (`dbg_pc`, `dbg_wb_addr`, `dbg_wb_data`, `dbg_wb_enable`). Nothing in simulation reads them. They exist because a module whose only ports are `clk` and `reset` has no primary outputs, so every net is unreachable and synthesis deletes the entire CPU as dead code. They are what a real trace port would carry anyway.

2.1 Supported Instruction Subset

Category	Instructions	Notes
R-type	<code>add</code> , <code>sub</code> , <code>and</code> , <code>or</code> , <code>slt</code>	funct7 bit 5 selects sub vs. add
I-type	<code>addi</code> , <code>andi</code> , <code>ori</code> , <code>slti</code>	Immediate arithmetic; 12-bit sign-extended
Load / Store	<code>lw</code> , <code>sw</code>	Word-aligned, computed base + offset
Branch	<code>beq</code> , <code>bne</code> , <code>blt</code>	One-hot branch control; resolved in ID

There is no separate ALU control unit. The main control unit decodes the 4-bit ALU operation directly from opcode, funct3 and funct7 bit 5, which narrows the ID/EX pipeline register: only the operation is carried forward rather than an `ALUOp` field plus the funct fields it would have to be combined with later.

2.2 System Architecture

The full datapath, with the four pipeline registers in teal, the hazard detection unit in green, the branch forwarding unit in orange and the ALU forwarding unit in violet. Every signal is named for the pipeline register it emerged from, so its prefix says which stage it lives in.

Reading the figure left to right: the PC and the instruction memory make up IF; decode, immediate generation, the register file read ports, the branch comparator and the branch target adder make up ID; the two forwarding muxes and the ALU make up EX; the data memory is MEM; and the write-back mux is WB. The three control units sit outside the datapath so that the datapath has exactly one source for every control signal it consumes.

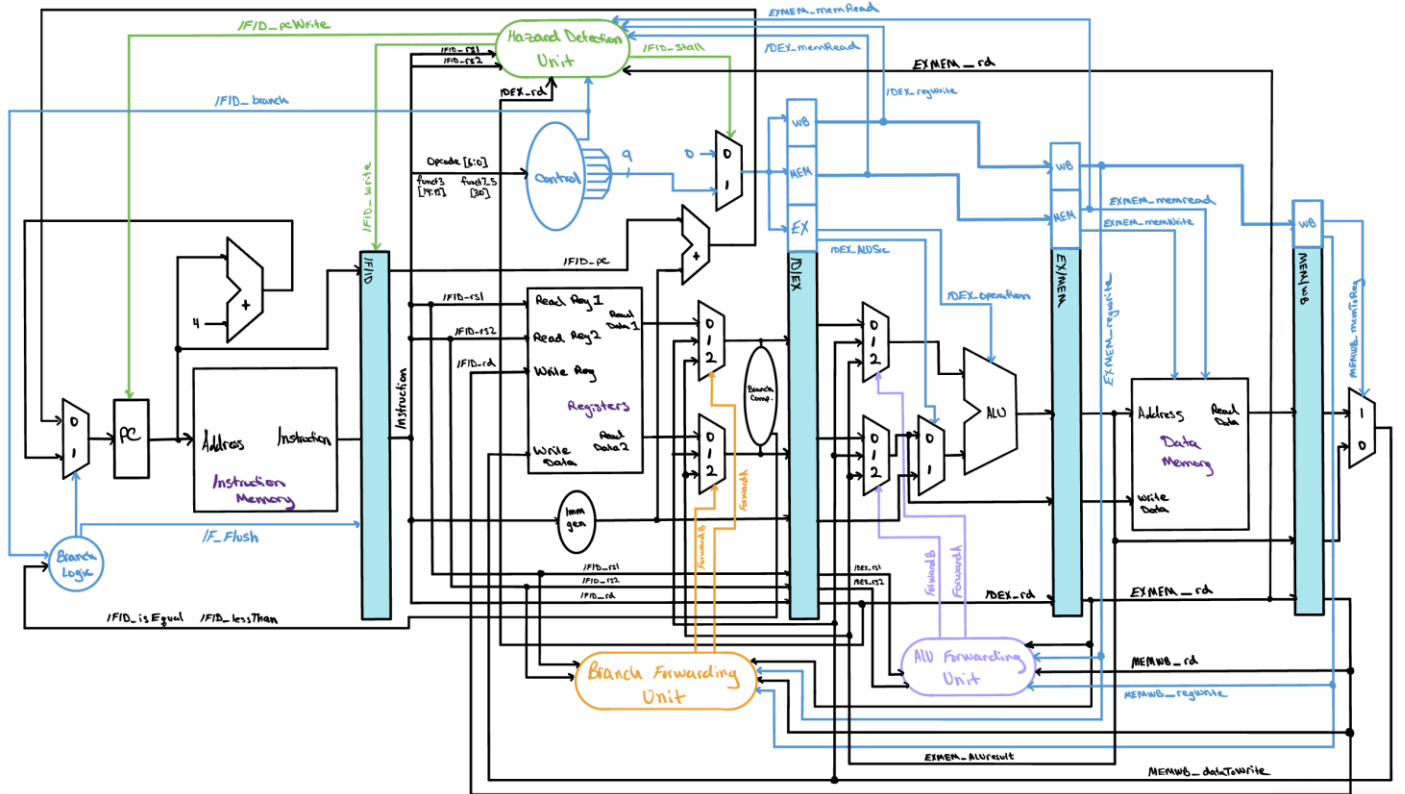


Figure 1: System Architecture, five-stage pipelined RISC-V CPU

2.3 Module Hierarchy

pipelined_cpu_top_level	
├ pipelined_cpu_control	Main control unit (combinational, decodes the ALU op)
├ hzrd_detection_unit	Load-use and branch-operand stall detection
├ alu_fwd_unit	EX-stage operand forwarding (from EX/MEM, MEM/WB)
├ branch_fwd_unit	ID-stage operand forwarding into the comparator
└ pipelined_cpu_datapath	
├ instruct_mem	Read-only instruction memory (loaded from program.mem)
├ ripple_carry_adder	PC + 4 adder (IF); PC + immediate adder (ID) (×2)
├ imm_gen	Immediate generator (I, S, SB, U, UJ formats)
├ reg_file	32×32-bit register file, negative-edge write
├ branch_comp	Equality and signed less-than, with forwarding muxes
├ mux_2x1	Branch/increment mux; ALU source mux; write-back mux (×3)
├ mux_4x1	ALU operand A and operand B forwarding muxes (×2)
├ alu_full	32-bit ALU (ripple-carry, parameterized)
├ alu_slice	1-bit ALU slice (bits [30:0])
├ mux_2x1	Operand invert select, a and b (×2)
├ mux_4x1	Operation select (and, or, add/sub, less)
└ full_adder	1-bit full adder
├ alu_msb	MSB ALU slice (overflow detection, setless)
├ mux_2x1	Operand invert select, a and b (×2)
├ mux_4x1	Operation select (and, or, add/sub, less)
└ full_adder	1-bit full adder (MSB, feeds overflow logic)
└ data_mem	Clocked data memory (word-aligned, read/write enable)

2.4 Pipeline Registers

Four registers separate the five stages. They are the only state the single-cycle design does not have, and they are where its 33% area difference comes from. The widths below are the declared widths in `pipelined_cpu_datapath.v` and come to 328 bits in all. Section 7.1 counts 325 flip-flops for them after synthesis, three fewer, because a bit the mapper can prove constant never becomes state.

Register	Width	Written at the end of	Carries
IF/ID	64 bits	IF	Instruction word and the PC of the instruction. Held on a stall, cleared on a flush.
ID/EX	120 bits	ID	Both register reads, the immediate, rs1, rs2, rd, and every control signal EX, MEM and WB will need. Control fields are zeroed to insert a bubble.
EX/MEM	73 bits	EX	ALU result, the store data, rd, and the MEM and WB control signals. Its ALU result is one of the two forwarding sources.
MEM/WB	71 bits	MEM	Loaded word, ALU result, rd, and the WB control signals. Its write-back data is the other forwarding source.

2.5 Hazard and Forwarding Units

Unit	Stage served	What it resolves
alu_fwd_unit	EX	Read-after-write on an ALU operand. Compares IDEX_rs1/rs2 against EXMEM_rd and MEMWB_rd; EX/MEM wins when both match, being the more recent result. Writes to x0 are ignored.
branch_fwd_unit	ID	The same comparison one stage earlier, feeding the branch comparator. Needed only because branches resolve in ID. EX/MEM is forwarded without qualifying on memRead, because a load in MEM feeding a branch has already been stalled by the hazard unit and cannot reach this comparison.
hzrd_detection_unit	ID	The three hazards forwarding cannot cover: a load in EX feeding the next instruction, any register-writing instruction in EX feeding a branch, and a load in MEM feeding a branch. It holds the PC and IF/ID and zeroes the ID/EX control fields.

A load immediately before a branch stalls twice, once from the ID/EX condition and again from the EX/MEM condition as the load advances. No counter and no state machine are involved: the two-cycle stall emerges from two consecutive single-cycle detections. The program exercises exactly this case at steps 16 and 17.

Only IF/ID is ever flushed. Because branches resolve in ID, nothing wrongly issued gets past ID, so IDEX_flush and EXMEM_flush are tied low. The flush is qualified with the stall: while the hazard unit is stalling a branch that is waiting on an operand, the take-branch decision is computed from data that is not ready, and the IF/ID register tests its flush path before its write path. An unqualified flush would clear IF/ID on a stall cycle and destroy the branch instruction itself.

3. Verification Methodology and Testbench

The testbench (tb_pipelined_cpu_top_level) is a self-checking, top-level integration testbench targeting Icarus Verilog (iverilog -g2012). It uses three independent oracles, so a bug in the design or in any one piece of checking code is unlikely to escape.

It is not the single-cycle testbench with a new DUT name, and it could not be. That testbench assumes one instruction commits per clock edge and that the fetch PC is the PC of the instruction committing. Neither holds here. An instruction retires four stages after it is fetched, the fetch PC runs up to four instructions ahead of the one committing, and a stall or a flush means some clock edges commit nothing at all.

3.1 Key Parameters

Parameter	Value / Purpose
CLK_PERIOD	10 ns — drives the always block that toggles clk
NUM_INSTR	30 — instructions in the program image, and the halt boundary
DMEM_WORDS	256 — number of 32-bit data-memory words compared every retirement
IMEM_WORDS	256 — reference instruction-memory size, mirroring the DUT
MAX_CYCLES	2000 — runaway watchdog limit, in clocks rather than in retirements
DRAIN_CYCLES	8 — clocks run on after the halt to prove nothing commits late

3.2 The Shadow Pipeline

Checking a pipeline needs an answer to a question a single-cycle testbench never has to ask: which instruction is committing this cycle? The testbench answers it with four registers of its own, carrying a PC and a valid bit alongside the DUT's four pipeline registers and clocked by exactly the same enable and clear conditions the RTL uses.

The valid bit is cleared, not merely left stale, on a flush or a stall, so a bubble is never mistaken for an instruction. It is also cleared for any fetch past the end of the program image, because instruct_mem returns a real NOP (0x00000013) for every unloaded word rather than nothing at all, and those NOPs would otherwise look like instructions retiring.

The conditions are copied from pipelined_cpu_datapath.v deliberately and have to stay in step with it. This is the testbench's model of where each instruction is, and it is only trustworthy if it advances on the same terms as the hardware.

3.3 Sampling Point

State is sampled just after the negative edge of the cycle in which an instruction sits in MEM/WB. By then both of its architectural effects are visible: a store wrote data memory at the positive edge that entered MEM/WB, and a register write lands at the negative edge of this cycle, because the register file is deliberately write-first-half and read-second-half.

3.4 Oracle 1: Lockstep Golden Model

A behavioral RV32I-subset simulator (task ref_step) decodes and executes one instruction per call, maintaining its own ref_regs[0:31], ref_dmem[0:255] and ref_pc, and encoding the correct architectural semantics. It is stepped once per retirement rather than once per clock, so stalls

and squashed instructions cost cycles in the testbench exactly as they do in the hardware. After every retirement the task `check_state` compares all 32 register-file entries and all 256 data-memory words against the DUT via hierarchical references.

The fetch PC is deliberately not compared. `dut.datapath.IF_pc` belongs to the instruction being fetched, which is up to four ahead of the one retiring. The equivalent check for a pipeline is the next oracle.

3.5 Oracle 2: Retirement Order

The task `check_retire_pc` compares the PC of the instruction leaving MEM/WB, read out of the shadow pipeline, against the PC the golden model just executed. This is the check a pipeline needs and a single-cycle CPU does not. A missing branch flush shows up here on the very first taken branch, as the instruction behind the branch retiring when the model has already moved to the branch target. A lost stall or an instruction committing twice shows up the same way.

3.6 Oracle 3: Final-State Table

The tasks `load_expected` and `check_final` encode a hand-derived table of the expected end-of-program register and memory state, checked once after halt. It is derived from the assembly listing in `programs/program_mem.txt` and cross-checked against a separate behavioral model, not read out of the DUT: copying a simulation result into it would turn the oracle into an expensive way of asserting that the DUT equals itself.

The table is unchanged from the single-cycle testbench, and that is the point. Pipelining must not change the architectural result, so the two designs are held to one identical expectation. Two of its entries carry most of the weight: `x1` and `x2` are the registers the two squashed instructions would have set to 99, so if a branch flush ever stops working they say so.

3.7 Pipeline Drain Check

After the halt the testbench snapshots the register file and data memory, clocks the design for a further `DRAIN_CYCLES` cycles, and compares. A single-cycle CPU cannot fail this way. A pipeline can: an instruction that should have been squashed is still in flight when the program ends and writes its result a few cycles later, long after every per-retirement check has passed.

3.8 Supported Instruction Subset of the Golden Model

Opcode group	Instructions	Notes
OPC_RTYPE (0110011)	add, sub, and, or, slt	funct7[5] selects sub; signed compare for slt
OPC_ITYPE (0010011)	addi, andi, ori, sli	12-bit sign-extended immediate
OPC_LOAD (0000011)	lw	Byte address = <code>rs1 + imm_i</code> ; word-aligned
OPC_STORE (0100011)	sw	Byte address = <code>rs1 + imm_s</code> ; word-aligned
OPC_BRANCH (1100011)	beq, bne, blt	PC-relative; signed compare for blt

3.9 Halt Condition and Watchdog

The execution loop terminates when the golden model's PC leaves the loaded program image (`ref_pc[31:2] >= NUM_INSTR`). The loop runs one iteration per clock and steps the model only on cycles the DUT actually commits something, so it cannot run ahead of the hardware. A global watchdog triggers \$finish after `CLK_PERIOD * (MAX_CYCLES + 50)` nanoseconds, and the loop itself also exits on `cycle_count >= MAX_CYCLES`, so a design that never retires anything terminates rather than hanging the simulator.

3.10 Waveform Dump

The testbench calls `$dumpfile("waveforms/dump.vcd")` and `$dumpvars(0, tb_pipelined_cpu_top_level)` in an initial block, producing a Value Change Dump covering the full hierarchy including the shadow pipeline. The VCD is viewable in GTKWave or in EPWave on EDA Playground. Appendix A is a capture of it.

3.11 Per-Instruction Trace and Disassembler

After each retirement the task `trace_print` emits a line carrying the retirement number, the cycle it retired in, the PC, the instruction word, a pseudocode disassembly produced by the task `disasm`, and a plain-English description of the architectural effect. The cycle number is the addition the single-cycle trace did not need: the gaps between consecutive cycle numbers are the stalls and flushes, and Section 4.2 reads them straight off this column. The `disasm` task covers every opcode in the subset and falls back to `.word 0XXXXXXXXX` for anything else, so an unexpected encoding is printed rather than silently mislabeled.

3.12 Hierarchical References

The testbench reaches into DUT state directly by dot notation, and these paths must match the instance names in the RTL:

<code>dut.datapath.registers.RF[k]</code>	– register file array (<code>k = 0..31</code>)
<code>dut.datapath.data_memory.memory[k]</code>	– data memory array (<code>k = 0..255</code>)
<code>dut.datapath.IF_pc</code>	– fetch PC, reported but never compared
<code>dut.IFID_write / dut.IFID_flush</code>	– shadow pipeline enables
<code>dut.IFID_stall / dut.IDEX_flush</code>	– shadow pipeline clears
<code>dut.EXMEM_flush</code>	– shadow pipeline clear

The last four are the ones that make this testbench pipeline-specific. If the RTL renames a flush or an enable, the shadow pipeline stops tracking the hardware and the retirement-order oracle starts reporting failures that are the testbench's, not the design's.

3.13 Verification Task Reference

Task	Role
<code>disasm</code>	Decode one 32-bit instruction word into readable pseudocode.
<code>ref_step</code>	Decode and execute one instruction in the golden model.
<code>check_state</code>	Compare all 32 registers and all 256 data-memory words.
<code>check_retire_pc</code>	Compare the retiring PC against the one the model just executed.
<code>trace_print</code>	Emit a per-retirement trace line, including the cycle number.
<code>drain_and_check</code>	Clock on after the halt and prove nothing commits late.
<code>dump_final_state</code>	Print the register file, the fetch PC, and non-zero memory.

Task	Role
load_expected	Populate the hand-derived final-state table.
check_final	Compare DUT final state against that table.
report_pipeline_stats	Print retirements, cycles, stalls, flushes and cycles per retire.

4. Program Under Test

The 30-instruction program (program.mem) is loaded into both the DUT instruction memory and the reference model at simulation start via \$readmemh. It is generated by programs/build_program.py, which holds the instruction list in source form and writes the identical image into this design and the single-cycle design, so the two cannot drift apart. Do not hand-edit program.mem.

Roughly half the program was written for this design rather than for the single-cycle one. The load followed by a use of its result, the load followed by a branch on its result, and the three-iteration backward loop all exist to force hazards that a single-cycle CPU cannot have.

4.1 Execution Trace

The cycle column is the cycle the instruction retired in. Where it jumps by more than one, the missing cycles are stalls or flushes and are accounted for in Section 4.2.

#	Cyc	PC	Encoding	Pseudocode	Effect
1	4	0x00000000	0x00500093	addi x1, x0, 5	x1 ← 0x00000005 (5)
2	5	0x00000004	0xFFD00113	addi x2, x0, -3	x2 ← 0xFFFFFFF (-3), negative immediate
3	6	0x00000008	0x001101B3	add x3, x2, x1	x3 ← 0x00000002 (2), both operands forwarded
4	7	0x0000000C	0x40308233	sub x4, x1, x3	x4 ← 0x00000003 (3), x3 forwarded from EX/MEM
5	8	0x00000010	0x001172B3	and x5, x2, x1	x5 ← 0x00000005 (5)
6	9	0x00000014	0x00416333	or x6, x2, x4	x6 ← 0xFFFFFFFF (-1)
7	10	0x00000018	0x001123B3	slt x7, x2, x1	x7 ← 0x00000001 (1), -3 < 5 signed
8	11	0x0000001C	0xFF0A413	slti x8, x1, -1	x8 ← 0x00000000 (0), 5 < -1 false
9	12	0x00000020	0x0030F493	andi x9, x1, 3	x9 ← 0x00000001 (1)
10	13	0x00000024	0x0080E513	ori x10, x1, 8	x10 ← 0x0000000D (13)
11	14	0x00000028	0x04000593	addi x11, x0, 64	x11 ← 0x00000040 (64), base address
12	15	0x0000002C	0x00A5A023	sw x10, 0(x11)	dmem[16] ← 0x0000000D
13	16	0x00000030	0x0005A603	lw x12, 0(x11)	x12 ← 0x0000000D (13)
14	18	0x00000034	0x001606B3	add x13, x12, x1	x13 ← 0x00000012 (18), after a load-use stall
15	19	0x00000038	0x00D5A223	sw x13, 4(x11)	dmem[17] ← 0x00000012
16	20	0x0000003C	0x0045A703	lw x14, 4(x11)	x14 ← 0x00000012 (18)
17	23	0x00000040	0x00D70463	beq x14, x13, +8	TAKEN → 0x00000048, after two stalls
—	—	0x00000044	0x06300093	addi x1, x0, 99	squashed by the flush at retirement 17
18	25	0x00000048	0x00400793	addi x15, x0, 4	x15 ← 0x00000004 (4)
19	26	0x0000004C	0x00900813	addi x16, x0, 9	x16 ← 0x00000009 (9)
20	28	0x00000050	0x0107C463	blt x15, x16, +8	TAKEN → 0x00000058, after one stall
—	—	0x00000054	0x06300113	addi x2, x0, 99	squashed by the flush at retirement 20

#	Cyc	PC	Encoding	Pseudocode	Effect
21	30	0x00000058	0x00F84463	blt x16, x15, +8	not taken, 9 < 4 false
22	31	0x0000005C	0x00F80463	beq x16, x15, +8	not taken, 9 == 4 false
23	32	0x00000060	0x00300893	addi x17, x0, 3	x17 ← 0x00000003 (3), loop counter
24	33	0x00000064	0x01190933	add x18, x18, x17	x18 ← 0x00000003 (3), loop pass 1
25	34	0x00000068	0xFFFF88893	addi x17, x17, -1	x17 ← 0x00000002 (2)
26	36	0x0000006C	0xFE089CE3	bne x17, x0, -8	TAKEN → 0x00000064, after one stall
27	38	0x00000064	0x01190933	add x18, x18, x17	x18 ← 0x00000005 (5), loop pass 2
28	39	0x00000068	0xFFFF88893	addi x17, x17, -1	x17 ← 0x00000001 (1)
29	41	0x0000006C	0xFE089CE3	bne x17, x0, -8	TAKEN → 0x00000064, after one stall
30	43	0x00000064	0x01190933	add x18, x18, x17	x18 ← 0x00000006 (6), loop pass 3
31	44	0x00000068	0xFFFF88893	addi x17, x17, -1	x17 ← 0x00000000 (0)
32	46	0x0000006C	0xFE089CE3	bne x17, x0, -8	not taken, loop exits, after one stall
33	47	0x00000070	0x07B00013	addi x0, x0, 123	x0 write discarded (I-type)
34	48	0x00000074	0x00208033	add x0, x1, x2	x0 write discarded (R-type)

Instructions at PCs 0x44 and 0x54 never retire. In the single-cycle design they were simply skipped by the taken branch; here they are fetched, enter IF/ID, and are cleared out of it by the flush before they can decode. Both write 99 to a register the final-state oracle checks, x1 and x2, so a broken flush is caught twice over: once by the retirement-order oracle at the branch itself, and again by the end-of-program table.

4.2 Cycle Accounting

48 cycles for 34 retirements. Every cycle that retired nothing is one of the following, and they sum exactly.

Cycles	Event	Cause
3	Pipeline fill	The first instruction is fetched in cycle 1 and retires in cycle 4.
1	Load-use stall	Retirement 13 loads x12; retirement 14 adds it. The loaded word is not available until the end of MEM, so it cannot be forwarded to the ALU in time.
2	Load-then-branch stall	Retirement 16 loads x14; retirement 17 branches on it. One stall from the ID/EX condition, then a second from the EX/MEM condition as the load advances and EX/MEM holds the address rather than the data.
4	ALU-then-branch stall	Retirements 20, 26, 29 and 32 each branch on a value the instruction directly ahead of them is still computing in EX. One cycle each.
4	Branch flush	The taken branches at retirements 17, 20, 26 and 29 each squash one wrongly fetched instruction. The not-taken branches at 21, 22 and 32 cost nothing.
48	Total	34 retirements + 3 fill + 7 stalls + 4 flushes.

Two things in that table are worth reading twice. Every stall in this program is caused by a branch except one, which is a direct consequence of resolving branches in ID: it buys back three of the four flush cycles a resolve-in-EX design would pay, and spends some of the saving on operands that arrive one stage too late. And a not-taken branch is free, so the loop exit at retirement 32 costs a stall but no flush.

5. Verification Results

5.1 Lockstep Check

The lockstep check fires after every retirement and compares all 32 registers and all 256 data-memory words against the golden model. Zero mismatches were recorded across all 34 retirements.

5.2 Retirement Order

The retiring PC matched the golden model's on all 34 retirements, including the four taken branches and the seven stalled cycles. No instruction committed twice and none committed out of order.

5.3 Pipeline Drain

Nothing changed in the register file or in data memory during the eight cycles clocked after the halt. No instruction was left in flight that should have been squashed.

5.4 Final-State Oracle

The hand-derived table was compared against the DUT final state after the drain. Zero mismatches. Every value in the two tables below is also the value the single-cycle design produced.

5.5 Final Register File

Register	Expected (hex)	Observed (hex)	Comment
x0	0x00000000	0x00000000	Hardwired zero; writes at retirements 33 and 34 discarded
x1	0x00000005	0x00000005	NOT 99, proving the flush at retirement 17 worked
x2	0xFFFFFFFF	0xFFFFFFFF	NOT 99, proving the flush at retirement 20 worked
x3	0x00000002	0x00000002	add x3, x2, x1 (-3 + 5), both operands forwarded
x4	0x00000003	0x00000003	sub x4, x1, x3 (5 - 2), x3 forwarded from EX/MEM
x5	0x00000005	0x00000005	and x5, x2, x1 (-3 & 5)
x6	0xFFFFFFFF	0xFFFFFFFF	or x6, x2, x4 (-3 3)
x7	0x00000001	0x00000001	slt x7, x2, x1, signed, true
x8	0x00000000	0x00000000	slti x8, x1, -1, signed, false
x9	0x00000001	0x00000001	andi x9, x1, 3
x10	0x0000000D	0x0000000D	ori x10, x1, 8

Register	Expected (hex)	Observed (hex)	Comment
x11	0x00000040	0x00000040	addi x11, x0, 64, data memory base
x12	0x0000000D	0x0000000D	lw x12, 0(x11), reads back the store at retirement 12
x13	0x00000012	0x00000012	add x13, x12, x1, after the load-use stall
x14	0x00000012	0x00000012	lw x14, 4(x11), reads back the store at retirement 15
x15	0x00000004	0x00000004	addi x15, x0, 4
x16	0x00000009	0x00000009	addi x16, x0, 9
x17	0x00000000	0x00000000	Loop counter, decremented to zero
x18	0x00000006	0x00000006	Loop sum: 3 + 2 + 1
x19–x31	0x00000000	0x00000000	Not written

The fetch PC at halt is 0x000000A4, four instructions past the last retirement at 0x00000074. That is the pipeline still fetching NOPs out of unloaded instruction memory while the last real instructions drain, and it is why the fetch PC is reported but never compared.

5.6 Final Data Memory (non-zero words)

Word index	Byte address	Expected (hex)	Observed (hex)	Source instruction
16	0x40	0x0000000D	0x0000000D	sw x10, 0(x11) at retirement 12
17	0x44	0x00000012	0x00000012	sw x13, 4(x11) at retirement 15

5.7 Pipeline Behavior

Metric	Value	Notes
Instructions retired	34	From a 30-instruction image: a three-iteration loop, less two squashed
Clock cycles to halt	48	Frozen before the drain, so the drain does not inflate it
Stall cycles	7	One load-use, six branch-operand
IF/ID flushes	4	One per taken branch
Cycles per retirement	1.41	48 / 34
Errors	0	Across all three oracles and the drain check

6. Functional Coverage

6.1 Coverage Matrix

Coverage Point	Status	Notes
R-type: add	COVERED	Retirements 3, 14, 24, 27, 30, 34
R-type: sub	COVERED	Retirement 4
R-type: and	COVERED	Retirement 5, negative operand
R-type: or	COVERED	Retirement 6, negative operand, result -1
R-type: slt (true)	COVERED	Retirement 7, -3 < 5 signed
R-type: slt (false)	NOT COVERED	Only the true case is exercised directly; slti covers the false path
I-type: addi (positive imm)	COVERED	Retirements 1, 11, 18, 19, 23, 33
I-type: addi (negative imm)	COVERED	Retirements 2, 25, 28, 31
I-type: andi / ori / slti	COVERED	Retirements 9, 10, 8
Load: lw	COVERED	Retirements 13, 16, both reading back a prior store
Store: sw	COVERED	Retirements 12, 15
Load-after-store, same address	COVERED	12/13 and 15/16
Branch: beq taken / not taken	COVERED	Retirements 17 / 22
Branch: bne taken / not taken	COVERED	Retirements 26, 29 / 32
Branch: blt taken / not taken	COVERED	Retirements 20 / 21
Backward branch / loop	COVERED	Retirements 24–32, three iterations
x0 write suppression, I and R	COVERED	Retirements 33 and 34
Forwarding: EX/MEM to EX	COVERED	Retirement 4 reads x3 one instruction after it was written
Forwarding: MEM/WB to EX	COVERED	Retirement 5 reads x1 two instructions after it was written
Forwarding: EX/MEM to ID (branch)	COVERED	Retirement 21 branches on x16 written two ahead
Forwarding: MEM/WB to ID (branch)	COVERED	Retirement 22 branches on x15 and x16
Forwarding priority, EX/MEM over MEM/WB	NOT COVERED	No instruction has the same register pending in both stages at once
Stall: load-use	COVERED	Retirement 14, one cycle
Stall: ALU result into a branch	COVERED	Retirements 20, 26, 29, 32
Stall: load into a branch (two cycles)	COVERED	Retirement 17
Flush: taken branch	COVERED	Retirements 17, 20, 26, 29
Flush qualified by stall	COVERED	Retirement 17 stalls and then flushes; the branch survives
Late write after halt	COVERED	Drain check, 8 cycles, no change

Coverage Point	Status	Notes
Registers x19–x31 written	NOT COVERED	Only x0–x18 are touched
Data memory outside words 16–17	NOT COVERED	Only two words are written

6.2 Squashed Instructions

Two instructions are fetched and never retire: `addi x1, x0, 99` at PC 0x44 and `addi x2, x0, 99` at PC 0x54. Their targets were chosen deliberately. x1 and x2 are read by the final-state oracle, so a flush that stopped working would be caught by the end-of-program table even if every per-retirement check somehow passed.

7. Comparison with the Single-Cycle Design

The two designs implement the same ISA subset and run the same program image, byte for byte, so they can be compared on equal terms. This section covers what changed in the RTL, what it cost in cycles, and what static timing analysis says it bought in clock period.

7.1 What Changed in the RTL

Aspect	Single-cycle	Pipelined
Stages	One; every instruction completes in a clock	Five, separated by four pipeline registers
ALU decode	Separate alu_control unit reading funct3 and funct7	Folded into the main control unit, which emits the 4-bit operation in ID
Branch resolution	In the same cycle as everything else	In ID, one stage after fetch, so one instruction is squashed per taken branch
Register file write	Positive clock edge	Negative clock edge, so a WB write is readable by an ID read in the same cycle
Data hazards	Cannot occur	alu_fwd_unit, branch_fwd_unit and hzrd_detection_unit
Control hazards	Cannot occur	IFID_flush, qualified with IFID_stall
Flip-flops after synthesis	1024	1349

The flip-flop counts are worth unpacking, because they are almost entirely attributable. Both designs contain 992 DFF_X1 cells, which are the 31 writable registers of the register file at 32 bits each. The single-cycle design adds 32 resettable flops for the PC, and nothing else has state. The pipelined design adds 357: the same 32-bit PC, and 325 bits of pipeline register.

7.2 Architectural Equivalence

Pipelining is only correct if it is invisible from the outside, and the verification is arranged so that this is checked rather than assumed. The final-state table in the pipelined testbench is the same table used by the single-cycle testbench, unchanged. Both designs end with the same 19 register values and the same two non-zero data-memory words. Every entry in Section 5.5 and Section 5.6 is therefore a cross-design comparison as well as a self-check.

7.3 Cycle Count

The single-cycle design retires 34 instructions in 34 cycles, one per cycle, which is the defining property of the style. The pipelined design retires the same 34 in 48. The 14-cycle difference is the fill, the stalls and the flushes itemized in Section 4.2.

That is the trade in its plainest form. A pipeline does not execute fewer instructions and on a short program it does not even execute them in fewer cycles. It executes them in shorter cycles, and it has to win by enough on the clock to pay for the cycles it added. How much it needs is arithmetic: $48 / 34$, or 1.41x.

7.4 Static Timing Analysis Flow

Cycle counts are what the testbenches measure. They cannot say how long a cycle is, so both designs were synthesized to standard cells and their critical paths measured. The flow lives in Verilog/CPU/sta and runs with a single command:

```
cd Verilog/CPU/sta
./run_synth.sh
```

yosys reads each design and runs generic synthesis; abc does technology mapping onto the Nangate45 45 nm typical library, sizes the mapped gates, and traces the longest register-to-register path cell by cell. Both designs are given the identical driver and load through abc_constr, a BUF_X1 driving cell and a 5 ff load, which is what makes the two numbers comparable. Without the constraint file abc stops after mapping, never runs stime, and reports no delay at all.

The two memories are read as blackboxes rather than as RTL, and that is not a shortcut. instruct_mem has no write port and fills its array in an initial block, so synthesis sees a constant ROM; worse, yosys ignores the \$readmemh because the NOP fill loop in the same initial block wins, so every word reads 0x00000013, which is addi x0, x0, 0. A CPU whose entire instruction stream writes only to x0 never writes its register file, so constant propagation collapses the datapath and leaves 34 flip-flops: the program counter, and nothing else. Even with the program correctly loaded, a ROM holding one fixed program lets synthesis specialize the CPU for that program, which measures the program rather than the processor. A real instruction memory is an SRAM macro whose contents are unknown at synthesis time, and the blackbox is the honest model of one.

The same reasoning is why both top levels carry the dbg_* outputs described in Section 2. Without primary outputs there is nothing to keep the logic alive and synthesis deletes the CPU entirely.

7.5 Measured Critical Path and Area

Memory access time is the one estimated input in the whole flow; everything else in the table is measured. The estimate is added afterwards, and it is added asymmetrically, which is most of the story: the single-cycle critical path crosses both memories inside one clock, while no pipeline stage crosses more than one.

Quantity	Single-cycle	Pipelined	Source
Logic critical path	2173 ps	1813 ps	Measured by abc on Nangate45 typical
+ instruction memory	700 ps	in IF	Estimated, 256x32 SRAM macro at 45 nm
+ data memory	600 ps	in MEM	Estimated, same
Clock period	3473 ps	1813 ps	Sum for single-cycle; worst single stage for pipelined
Maximum frequency	288 MHz	552 MHz	1 / clock period
Cell area	10651 μm^2	14153 μm^2	yosys stat against the Liberty file, +33%
of which sequential	4656 μm^2 (43.7%)	6385 μm^2 (45.1%)	Flip-flops only

The area growth is 3502 μm^2 , and it splits almost evenly: 1729 μm^2 of it is the pipeline flip-flops themselves and 1773 μm^2 is combinational, which is the two forwarding units, the hazard

detection unit, the two 4x1 forwarding muxes on the ALU inputs, the ID-stage branch comparator with its own forwarding muxes, and the extra fan-out of control signals that now have to reach four stages instead of one.

7.6 Runtime and Speedup

Quantity	Single-cycle	Pipelined
Clock period	3473 ps	1813 ps
Cycles for the program	34	48
Runtime	118.1 ns	87.0 ns
Clock ratio needed to break even	—	1.41x
Clock ratio achieved	—	1.92x
Speedup	—	1.36x

The pipeline needed a 1.41x clock advantage to pay for its extra 14 cycles and it got 1.92x, so the program finishes 1.36x sooner. The margin between 1.41 and 1.92 is the whole return on the design, and on a longer program it would be larger: the three fill cycles are a fixed cost that a 34-retirement program pays in full and a 34,000-retirement program does not notice.

7.7 Sensitivity to the Memory Estimate

Because the memory access time is estimated rather than measured, and because it is added to only one of the two designs, the conclusion should be tested against it. The flow takes the two values from the environment, so this is one command each.

T_IMEM / T_DMEN	SC period	PL period	SC runtime	PL runtime	Speedup
300 / 300 ps	2773 ps	1813 ps	94.3 ns	87.0 ns	1.08x
700 / 600 ps	3473 ps	1813 ps	118.1 ns	87.0 ns	1.36x
1000 / 900 ps	4073 ps	1813 ps	138.5 ns	87.0 ns	1.59x

The pipelined clock period does not move at all across that range, because at every one of these values its 1813 ps logic path is still the slowest stage and the memories are hidden inside stages that are not critical. The single-cycle period tracks the estimate directly. The pipeline wins in every case, and it wins by more as memories get slower, which is the direction real memories have moved relative to logic for forty years.

7.8 What the Flow Does Not Model

The number reported is the longest register-to-register logic path with real cells, real gate sizing and real pin capacitances. It is not full static timing analysis. Wire delay, clock skew, setup and hold margining, and clock-aware path checking are all absent.

One omission is specific to this design and worth naming. The pipelined register file writes on the negative clock edge, so every write-back path has half a clock rather than a full one to close. abc has no concept of a clock edge and cannot see this. The path passes comfortably at the periods above, but that is inference from the logic delay rather than a proof. sta/run_sta.sh exists to run the same netlists through OpenSTA, which is clock-aware; OpenSTA has no Windows build, so that half of the flow needs WSL and is currently untested.

8. Issues and Recommendations

8.1 Open Issues

#	Issue	Details
1	Forwarding priority untested	No instruction in the program has the same register pending in EX/MEM and MEM/WB simultaneously, so the rule that EX/MEM wins is never exercised. A three-instruction sequence writing and then twice reading one register would close it.
2	Upper registers unexercised	Registers x19 through x31 are never written, so the upper half of the register file address decode is untested in both designs.
3	Data memory sparsely exercised	Only word indices 16 and 17 are written. Index 0 and the upper boundary at index 255 would stress the address decode.
4	Half-cycle path unproven	The negedge register-file write creates a half-cycle write-back path that the abc flow cannot check. It needs OpenSTA, which needs WSL.
5	Memory timing is estimated	T_IMEM and T_DMEN are the only non-measured inputs to Section 7. Section 7.7 bounds the effect, but a real macro datasheet would remove the estimate.

8.2 Recommendations

- Add a forwarding-priority sequence to programs/build_program.py: write a register, then read it from the two following instructions, so EX/MEM and MEM/WB are both candidates on the same cycle.
- Extend the program into the upper register file and to data-memory word 0 and word 255. Because build_program.py writes both designs' images, this closes the same gap in the single-cycle report.
- Add a fault-injection run: force one pipeline register to a wrong value mid-simulation and confirm that the retirement-order oracle flags it on the next retirement rather than at the end.
- Install WSL and run sta/run_sta.sh so the negedge write-back path is checked by a clock-aware tool rather than inferred.
- Add a `timescale directive to each RTL module to remove the Icarus Verilog timing-precision warning that appears when mixed-timescale designs are compiled together.

9. Conclusion

The five-stage pipelined RISC-V CPU (pipelined_cpu_top_level) passes all verification checks on the 30-instruction test program, retiring 34 instructions in 48 clock cycles with zero errors. The lockstep model, the retirement-order oracle and the independently hand-derived final-state table all agree, and nothing committed during the eight-cycle drain that followed the halt.

The design correctly implements every instruction in the supported subset and, more to the point, correctly implements the machinery that a pipeline needs and a single-cycle design does not: forwarding from both later stages into EX and into the ID-stage branch comparator, stalls for the three hazards forwarding cannot cover, and a single-instruction flush on a taken branch that is correctly qualified so it cannot destroy the branch it serves.

The architectural result is identical to the single-cycle design's, which is the requirement, and the timing analysis shows the pipeline running its clock 1.92 times faster for 33% more cell area, finishing the same program 1.36 times sooner. The coverage gaps in Section 8.1 are gaps in the stimulus, not defects in the design, and each has a named fix.

Appendix A. Simulation Waveform

The capture below covers the whole run: 34 instructions retired in 48 clock cycles. Signals are grouped by the pipeline register they belong to, which is also how they are named. Clock, reset and the fetch PC are at the top in yellow and violet; the IF/ID signals including takeBranch, stall and flush are in purple; the ID/EX signals including the two forwarding selects are in green; the EX/MEM signals in red; and the MEM/WB signals in orange.

The three control events of Section 4.2 are all directly visible. Each pulse on IFID_stall is a cycle where IF_pc and IFID_pc hold their value instead of advancing. Each pulse on IFID_flush follows a pulse on IFID_takeBranch by one cycle and clears IFID_instr. The load-use stall is the IFID_stall pulse that coincides with IDEX_memRead rather than with IFID_takeBranch.

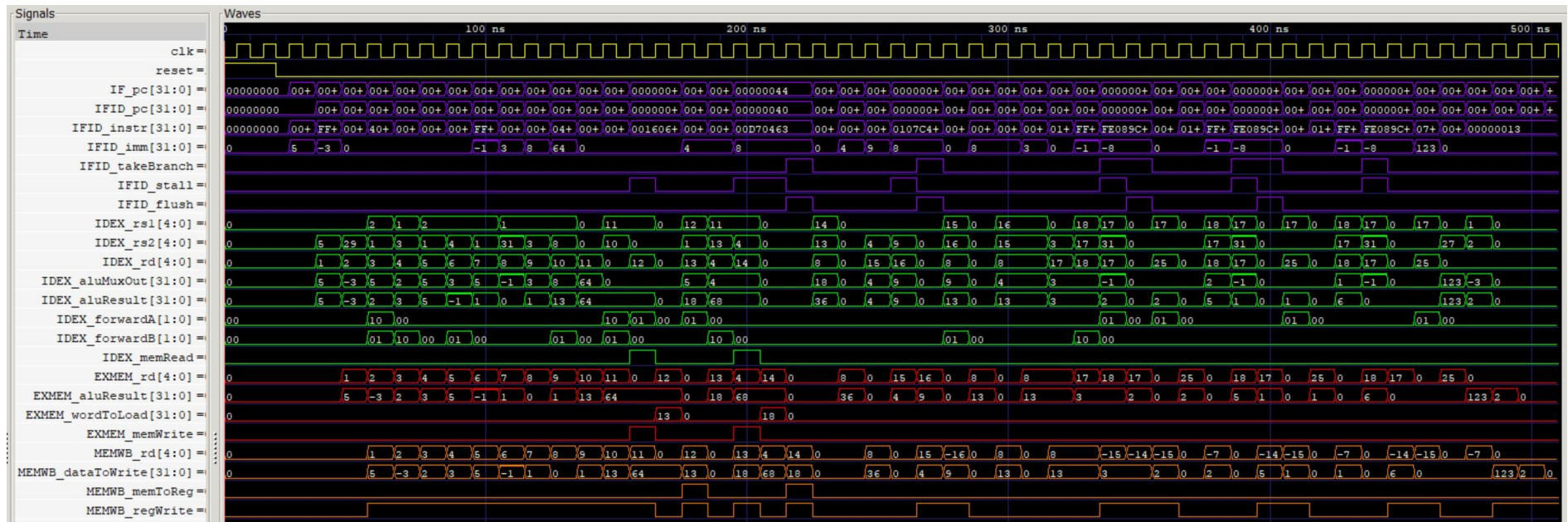


Figure 2: Full simulation waveform, 48 cycles